TELUSKO

AI Engineering Foundations - Part 1

Core AI and Machine Learning terminology for the Agentic AI with Java and Spring course

HOW TO READ THESE NOTES

This was the first technical class of the course. It is almost fully theory. The goal is not to master any single topic here but to get a clear mental picture of the words you will hear again and again once the coding starts. Take it slow, read once, and move on. Every one of these ideas comes back later in a practical, hands on form.

WHAT THIS SESSION COVERS

1 The real goal of the course

Before any terminology, it helps to know where all of this is going. This course is not about building a brand new ChatGPT or training a new AI model from scratch. That is a different world that needs heavy mathematics and months of deep study.

The actual focus is **AI Engineering** and **Agentic AI using Java and Spring Boot**. In simple words, you take AI models that already exist (from OpenAI, Anthropic, Google and others) and you bring their capabilities into your own applications, and later you build AI agents that can use tools and take actions.

> **KEY IDEA**
>
> You are the person who **uses** and **wires up** AI models inside real software. You are not the person building the model itself. Because of that, you only need a clear picture of the foundation terms, not a PhD in each one.

2 Traditional software versus an AI system

This is the single most important distinction in the whole session. At the end of the day both are just software running on a processor, but they behave very differently.

Traditional software

In traditional software, we (the developers) write explicit rules. The machine simply executes what we programmed. A classic example is a grading system:

```
if (marks >= 80)      grade = "Distinction";
else if (marks >= 35) grade = "Pass";
else                  grade = "Fail";
```

Here every rule is hand written. The output is **predictable**. If you enter 35, you know exactly what comes back. If you click "Enroll Now" on a website, you and the developer both know what will happen. There is no intelligence, only instructions being followed.

AI system

An AI system is not given the rules line by line. Instead we give it a large amount of data, it learns the patterns from that data, and then it makes predictions. The output is **not fully predictable**. If you ask an AI to "generate an image of a cat", nobody, not even the instructor, can tell you in advance exactly what that image will look like.

Think about it historically. Before AI, could you imagine a normal program writing a fresh poem on its own, or generating a brand new video from a sentence? To do that with plain if and else rules, you would need an impossible number of developers writing endless conditions. That is why we needed a different approach.

Traditional software	AI system
Rules are explicitly programmed by us	Patterns are learned from data
Output is predictable	Output is a prediction, not fixed
Example: check exam result by seat number	Example: generate a poem, image, or video
Behaves the same every time	Can respond to open, emotional, or new inputs

> **GOOD TO REMEMBER**
>
> The idea of "make a machine behave like a human" is old. It existed back in the 1950s to 1970s when programming was young. The idea was ready, but the hardware and infrastructure were not. What changed recently is the compute power, which finally let the idea become real.

3 What is AI, in plain words

There is no single perfect definition, so two official ones are worth keeping:

- **Google Cloud:** AI is a technology that enables computers and machines to simulate human learning, problem solving, creativity, and decision making.
- **OpenAI:** AI is a system that can perform tasks that typically require human intelligence.

The key word in both is **simulate**. We are not creating a human brain or a new human being. We are simulating human style intelligence in software.

> **THE ONE LINE THAT MATTERS**
>
> Every response you get from ChatGPT, Gemini, or Claude is not "thinking" and it is not really searching the internet in that moment. At its core it is doing **prediction**. Just like your phone keyboard predicts "you" after "how are", these models predict the next piece of text, only at a massive scale.

4 Machine Learning: data, features, labels

Machine Learning (ML) means training a machine on a large amount of data so that it learns the patterns and can then make predictions. It is the foundation under every AI system.

A simple human comparison: a small child does not recognise a cat the first time. After seeing many cats of different colours, sizes and breeds, the child's brain learns the pattern and can then say "that is a cat". A machine learns the same way, from many examples.

Three words to hold on to, using a house price example:

- **Data:** the raw records you feed in (many houses).
- **Features:** the characteristics of each record. For a house that is location, area in square feet, number of bedrooms, age of the house, and so on.
- **Label:** the answer attached to that record. For example, a 4 BHK, 3000 sq ft house in Richmond Town, Bengaluru has a label (price) of about 4 crore.

You feed thousands of such "features to label" examples. Slowly the machine learns the relationship, and later it can predict the price of a house it has never seen.

5 Three ways a machine learns

These are simply different training styles. Do not overthink them.

Type	What we give	Simple example
Supervised learning	Labelled data (features plus the correct answer)	House features with known prices, so it learns to predict price
Unsupervised learning	Only data, no labels. The machine finds groups by itself	Grouping millions of customers into budget, premium, frequent buyers. Netflix style recommendations
Reinforcement learning	Rewards for good moves, penalties for bad moves	The machine tries actions, gets plus points or minus points, and learns what is right

> **TAKEAWAY**
>
> All three are just different roads to the same place: a machine getting trained on data. That is the whole conclusion you need here.

6 Deep Learning and Neural Networks

Deep Learning is a subset of Machine Learning that is built on the idea of **neural networks**. Generative AI, the thing we all use today, is one of the applications that came out of deep learning.

Where the idea comes from

Researchers wanted software to learn the way humans learn. The human brain has billions of tiny cells called **neurons**, all connected and passing signals to each other. When you see a cat, millions of neurons fire together and instantly give the answer "cat", because your brain is already trained.

A neural network copies this idea. It is not a real brain. It uses artificial "neurons" called **nodes**, arranged in layers, that work together to spot patterns and make predictions.

The three layers

- **Input layer:** receives the data (for a house: location, bedrooms, area, age).
- **Hidden layer:** analyses the patterns and relationships in the data.
- **Output layer:** produces the final prediction (the price).

A **deep neural network** is simply a neural network with many hidden layers instead of one. That is the only difference between "neural network" and "deep".

Weights and the activation function

Not every input matters equally. For a house, location usually matters more than the age of the house. To represent this importance, the network assigns a **weight** to each input. A higher weight means that input has more influence on the final prediction.

> **IMPORTANT**
>
> These weights are **not** programmed by hand. The network learns them automatically during training. If you train it that a 4 BHK in Richmond Town is 5 crore and the same 4 BHK in a distant village is 1 crore, it works out on its own that location carries a high weight.

After combining inputs and weights, the neurons decide what information moves forward. That decision is handled by an **activation function**, which you can think of as a filter that decides what passes on to the next layer.

7 Natural Language Processing (NLP)

NLP is the part of AI that helps a computer understand, process, and generate human language. Human language here is not only English. It is Hindi, Kannada, Telugu, Tamil, Arabic, French, any natural language people use every day.

A processor only understands zeros and ones. You do not search Google in binary. You type "best pizza places near me". NLP is the bridge that sits between your **human language** and the **computer language**.

WHERE YOU ALREADY USE NLP

Google Search, ChatGPT, Gemini, chatbots, translation systems, voice assistants. Anything that takes your everyday sentence and acts on it is using NLP behind the scenes.

8 Tokenization and tokens

A computer cannot swallow a whole sentence at once. So the first job inside NLP is **tokenization**: breaking text into smaller pieces called **tokens**.

Think of a pizza. You do not eat it whole, you slice it. In the same way "I love Java programming" is broken into smaller units.

```
"I love Java programming"
   ->  [ "I" ] [ "love" ] [ "Java" ] [ "programming" ]
```

THE RULE TO REMEMBER

The entire AI ecosystem works on **tokens**, not on full sentences or paragraphs. The prompt you send in is tokens (input tokens). The reply you get back is tokens (output tokens). This is exactly why AI APIs charge you per token.

One subtle point raised in class: a token is not always one full word split by spaces. A tokenizer may split "programming" into "program" and "ming", or "unbelievable" into two or three tokens. It depends on the tokenizer that the model was trained with. Your brain splits on spaces out of habit, but the model does not have to.

TRY IT YOURSELF

A tokenizer playground lets you paste any text and see how it splits into tokens and IDs. OpenAI's tool is at `https://platform.openai.com/tokenizer` and the community "Tiktokenizer" is at `https://tiktokenizer.vercel.app`.

9 Embeddings and vectors

After tokens, each token is given an ID, and then a numerical representation. This is the core of how a computer "understands" words.

- **Embedding:** the process (and result) of converting text into numbers. Every word gets a numerical representation so the computer can work with its meaning.
- **Vector:** that list of numbers itself. For example the word "love" might be represented as `[0.3, 0.5]`. That list is its vector.
- **Dimensions:** how many numbers are in the vector. More dimensions means more information stored about that word, which usually means more accurate predictions.

```
tiger  ->  [0.81, 0.10, 0.55, 0.42, ...]   (many dimensions = height, colour,
breed, pattern...)
cat    ->  [0.79, 0.12, 0.50, ...]
car    ->  [0.05, 0.90, ...]
```

> **IN ONE LINE**
>
> Tokenization splits text into tokens. Embedding turns each token into numbers (a vector). More dimensions means the AI holds richer meaning about that word. You will store and use these in a **vector store** when the practical Spring AI classes begin.

10 Word2Vec and GloVe

Giving words numbers is one thing. The harder goal is to give numbers that **preserve meaning**, so related words sit close together and unrelated words sit far apart. This is where two well known techniques come in. Both belong to the NLP family and both are types of word embedding.

Bag of Words (the earlier idea)

Before these, there was a simpler "Bag of Words" approach. It gave numbers to words, but the numbers did not capture relationships well, so "cat" and "dog" could end up far apart even though they are both animals. The computer could not relate them.

Word2Vec

Word2Vec was developed by **Google**. Its core idea is beautifully simple: *words that appear in similar contexts tend to have similar meaning*. Because "the king lives in a palace", "the queen lives in a palace", "the prince lives in a palace" all appear in similar surroundings, king, queen and prince end up with similar vectors. Similar words move closer to each other.

Word2Vec trains in two ways:

- **CBOW (Continuous Bag of Words):** predict the centre word from the neighbours. Example: "I ___ Java", predict the missing middle word.
- **Skip-gram:** given one word, predict its neighbours. Example: given "love", predict "I" and "Java". Skip-gram is the more commonly used of the two.

The catch: Word2Vec looks at **local context** only, the nearby words. It is like learning about one street instead of the whole city.

GloVe (Global Vectors)

GloVe, short for Global Vectors, was developed at **Stanford University**. Instead of only nearby words, it looks at **global** word co-occurrence statistics across a huge collection of documents. If Word2Vec studies one street, GloVe studies the whole city.

	Word2Vec	GloVe
Developed by	Google	Stanford University
Context used	Local (nearby words)	Global (whole corpus statistics)
Family	Word embedding (NLP)	Word embedding (NLP)

> **THE LIMITATION OF BOTH**
>
> Both give **one fixed vector per word**. So "bank" gets a single vector, whether you mean "I deposited money in the bank" or "I sat near the river bank". They cannot tell the two meanings apart. That missing piece, context, is exactly what the next breakthrough solved.

Reference: GloVe project page, `https://nlp.stanford.edu/projects/glove/`

11 Transformers and Attention

In **2017** a research paper reshaped the entire AI world. It is called **"Attention Is All You Need"**. Every modern large language model (ChatGPT, Gemini, Claude and the rest) is built on the architecture from this paper: the **Transformer**.

Reference: "Attention Is All You Need", Vaswani et al., `https://arxiv.org/abs/1706.03762`

The problem it solved: context

Take the sentence: "*The animal did not cross the street because it was too tired.*" As a human you instantly know "it" means the animal, not the street. Your brain focuses on the important words. A computer with only fixed word vectors struggles with this. It cannot easily tell what "it" refers to.

Attention

Attention is the mechanism that lets the model focus on the important words in a sentence, the same way you highlighted key lines in a textbook while studying. It works out which words matter most for the meaning.

Self-attention

Self-attention looks at the other words in the same sentence to fix the meaning of each word. For "I deposited money in the bank", the nearby word "money" tells the model this "bank" is a financial institution. For "I sat near the river bank", the word "river" tells it this "bank" is a river side. Same word, different meaning, decided by context.

Multi-head attention

Multi-head attention means running several attention mechanisms in parallel, each learning a different relationship in the same sentence.

- Cricket broadcast analogy: instead of one camera, many cameras cover the batsman, the bowler, and the fielder at the same time. You get a richer view.
- For "Apple released a new iPhone in California", one head focuses on "Apple", another on "iPhone", another on "California". Together they capture much more than a single

focus could.

Encoder and Decoder

A transformer has two main parts. The simplest way to remember them:

Component	Role	Think of it as
Encoder	Reads and understands the meaning of the input	The reader
Decoder	Generates the output text word by word, based on that understanding	The writer

The overall flow when you send a prompt:

```
Input text
   -> Tokenization (split into tokens)
   -> Encoder (understand the context / meaning)
   -> Decoder (generate the output word by word)
   -> Output (the prediction you see)
```

On top of transformers sit families of models such as **BERT** (great for understanding tasks) and **GPT** (great for writing, chatting, coding, and generating). The LLM effectively "sits" at the decoding stage where the text is generated.

12 Frontier models, open source, and fine tuning

A useful side discussion from the class about the model landscape:

- **Frontier (hosted) models:** the strong paid models served by companies over an API, for example OpenAI's models, Google's Gemini, and Anthropic's Claude. In this course the instructor's personal ranking placed Claude at the top, then Google, then OpenAI, but for learning the concepts the choice does not matter. API prices have been rising over the last year.
- **Open source models:** models you can run on your own machine or server. On their own they cannot beat the frontier models. But if you **fine tune** a smaller local model

on your own data for your specific use case, it can outperform a frontier model *for that narrow task*, while keeping your data private and cutting cost.

- **Context window:** how much text a model can consider at once, measured in tokens. Large frontier models can reach very high context windows (the class mentioned figures like one million tokens for top models, versus around 128K tokens for a strong local model). Do not worry if this is fuzzy now, it becomes clear later.

> **ON FINE TUNING**
>
> You do not need to master the deep mathematics of machine learning, loss functions or NLP internals to fine tune a model today. Modern tooling and AI itself handle most of the heavy lifting. What you **do** need is to understand the concepts, so when the tools say "we adjusted this" you know which term they mean. The plan mentioned in class was to later run a local model on a dedicated AI server (a workstation class GPU) and even train a model to reply in the instructor's own style.

13 One page summary of the whole journey

Read this top to bottom and the whole session clicks into place:

1. **AI** = software that simulates human intelligence by learning from data and making predictions, instead of following hand written rules.
2. **Machine Learning** is how it learns: give it data (features to labels), it finds patterns. Styles: supervised, unsupervised, reinforcement.
3. **Deep Learning** is Machine Learning powered by **neural networks** (nodes in input, hidden, and output layers, with learned weights).
4. **NLP** lets computers understand human language.
5. NLP first does **tokenization** (split text into tokens).
6. Tokens become numbers through **embeddings**; the number lists are **vectors**.
7. **Word2Vec** (Google, local context) and **GloVe** (Stanford, global context) gave those vectors real meaning, but only one fixed vector per word, so no true context.
8. **Transformers** ("Attention Is All You Need", 2017) fixed this using **attention**, **self-attention**, and **multi-head attention**, with an **encoder** (reader) and **decoder** (writer).

9. Modern LLMs (GPT, BERT and others) are built on this transformer architecture. Everything you will build sits on top of these models.

FOR THE NEXT CLASS

No setup needed beyond a basic IDE and Java installed. From the next session the course shifts from theory to hands on coding, integrating real AI models into Java and Spring applications. Watch the five foundation videos in your dashboard if you have not yet.

TELUSKO Telusko | AI Engineering and Agentic AI with Java and Spring Boot. Notes for Session 01.